ADVANCED DATA STRUCTURES

JAVA • FUNDAMENTALS TO DEEP INTERVIEW & CP MASTERY

Tries • Segment Trees • Lazy Propagation • Fenwick Trees • Sparse Tables • DSU • Balanced Trees • Monotonic Structures • Suffix Arrays • Caches

Plus the important missing structures: heaps, hash-based structures, ordered maps, persistent trees, sqrt decomposition, offline queries, string matching, graph connectivity, and range-query patterns.

A practical reference: learn the model → derive the operation → trace the example → implement → analyze complexity → solve problems.

How to study this masterbook

This book is designed for demanding technical interviews and competitive programming. “JEE Advanced-like” here means rigorous reasoning and problem-solving depth—not that these exact structures form a JEE syllabus. Java is used for implementations. Examples prioritize clarity and correct invariants.

• Start with arrays, recursion, binary search, trees, heaps, hashing and graph basics.

• For each structure, understand what query/update it accelerates and what trade-off it introduces.

• Trace every implementation on a tiny example before using it on large input.

• Memorize invariants, not code. In interviews, explain why each operation is correct.

• Use long for large sums, counts, products or indices when constraints demand it.

Important scope note: Advanced data structures are a large field. This handbook covers the major interview/CP structures and patterns, but no single PDF can contain every specialized structure or every possible problem. It gives a broad core plus advanced extensions and a roadmap for deeper study.

Roadmap and contents

Part	Coverage
I	Foundations: complexity, arrays, hashing, heaps, trees
II	Trie: prefix, counting, deletion, bitwise trie
III	Segment Tree: build, query, point/range update, iterative form
IV	Lazy Propagation: range add/set, push/pull, invariants
V	Fenwick Tree: point update, prefix/range sums, range updates, 2D
VI	Sparse Table: idempotent queries, overlapping blocks
VII	DSU: compression, union-by-size, rollback, offline connectivity
VIII	Ordered Sets / Balanced BSTs: AVL, Red-Black, Treap, order statistics
IX	Monotonic Stack and Queue: next greater, histogram, sliding window
X	Suffix Array and string indexing: LCP, Kasai, applications
XI	LRU/LFU caches: hashmap + linked structures, O(1) design
XII	Additional structures: heap, sqrt decomposition, persistent tree, wavelet overview, HLD, Euler tour, Mo's algorithm
XIII	Pattern selection, complexity sheet, interview/CP problem roadmap

1. Foundations: choose the right structure

The query/update model

Before choosing a data structure, list the operations: point update? range sum? range minimum? prefix search? insert/delete keys? connectivity? prefix lookup? Then compare time and memory.

Need	Likely structure	Typical operations
Prefix lookup / dictionary	Trie / HashMap	Insert, search, startsWith
Range sum, point update	Fenwick Tree	O(log n) update/prefix
Range min/max, static	Sparse Table	O(1) query for idempotent op
Range query + updates	Segment Tree	O(log n)
Range update + range query	Lazy Segment Tree	O(log n)
Dynamic connectivity	DSU	Near O(1) amortized
Sorted dynamic keys / rank	Balanced BST / Treap	O(log n)
Next greater / window extrema	Monotonic stack/deque	O(n) total
Repeated substring queries	Suffix Array + LCP	O(n log n) build

Asymptotic notation

• O(1): constant; O(log n): tree-height or divide search; O(n): scan; O(n log n): sorting and many tree builds; $O(n^2)$: all pairs.

• Amortized complexity averages over a sequence (e.g. dynamic array append, DSU with compression).

• Expected complexity depends on randomness or hashing assumptions (e.g. hash table, randomized treap).

• Space complexity must include auxiliary arrays, recursion stacks and stored nodes.

Array, hashing, heap, tree quick review

Array gives O(1) indexing but insertion in the middle is O(n). HashMap gives expected O(1) lookup but no sorted order. Heap gives O(1) min/max peek and O(log n) insert/remove-root, but not arbitrary search. Balanced BST maintains sorted dynamic keys with O(log n) operations.

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
minHeap.offer(8);
minHeap.offer(3);
minHeap.offer(5);
int smallest = minHeap.poll(); // 3

Map<String, Integer> freq = new HashMap<>();
freq.put("cat", freq.getOrDefault("cat", 0) + 1);
```

2. Trie (Prefix Tree)

What problem does a Trie solve?

A Trie stores strings by shared prefixes. Each edge represents a character; a path from root spells a prefix. A terminal marker distinguishes a complete stored word from a prefix of a longer word.

```
Words: to, tea, ten, in, inn

(root)
  |t                |i
  |                 |
  o*                n*
  |                 |
  a*  e             n*
      | \
      a* n*
* = complete word
```

For alphabet size σ and word length L: insert/search/prefix lookup O(L), independent of the number of stored words in the usual fixed-alphabet model. Space can be large: O(total characters × child representation cost).

Java Trie for lowercase English letters

```
class Trie {
    static class Node {
        Node[] child = new Node[26];
        boolean end;
    }
    private final Node root = new Node();

    void insert(String word) {
        Node cur = root;
        for (char ch : word.toCharArray()) {
            int i = ch - 'a';
            if (cur.child[i] == null) cur.child[i] = new Node();
            cur = cur.child[i];
        }
        cur.end = true;
    }

    boolean search(String word) {
        Node n = walk(word);
        return n != null && n.end;
    }

    boolean startsWith(String prefix) {
        return walk(prefix) != null;
    }

    private Node walk(String s) {
        Node cur = root;
        for (char ch : s.toCharArray()) {
            int i = ch - 'a';
            if (i < 0 || i >= 26 || cur.child[i] == null) return null;
            cur = cur.child[i];
        }
        return cur;
    }
}
```

This implementation assumes lowercase a-z. For Unicode or arbitrary characters, use `Map` or a compressed/radix trie. Empty string is represented by root.end.

Counting words and prefixes

Add `passCount` to each node (number of inserted words passing through it) and `endCount` (number ending there). This supports duplicate words, prefix counts and deletion. On insert increment passCount along path and endCount at terminal.

```
static class CountNode {
    Map<Character, CountNode> next = new HashMap<>();
    int passCount, endCount;
}
// Insert outline:
// node=root; node.passCount++;
// for each char: node=child; create if missing; node.passCount++;
// node.endCount++;
```

Trie deletion

To delete one copy: first confirm endCount > 0; decrement endCount and passCount along the path. Remove child links bottom-up while a child has passCount == 0. Never remove a node that still supports another word.

Trie variants

• Compressed/Radix Trie: compress chains of single-child nodes; saves memory.

• Binary Trie: store bits of integers; supports max/min XOR queries in O(number of bits).

• Wildcard Trie search: DFS branching at wildcard positions; worst case exponential in wildcard count.

• Autocomplete: DFS from prefix node, ideally with cached top-k suggestions for fast ranked results.

3. Binary Trie: maximum XOR

For each bit from high to low, to maximize XOR with x, prefer the opposite bit if that child exists. If not, follow the same bit.

```
class BitTrie {
    static class N { N[] c = new N[2]; }
    N root = new N();

    void add(int x) {
        N n = root;
        for (int b = 31; b >= 0; b--) {
            int bit = (x >>> b) & 1;
            if (n.c[bit] == null) n.c[bit] = new N();
            n = n.c[bit];
        }
    }

    int maxXor(int x) {
        N n = root;
        int ans = 0;
        for (int b = 31; b >= 0; b--) {
            int bit = (x >>> b) & 1;
            int want = bit ^ 1;
            if (n.c[want] != null) {
                ans |= (1 << b);
                n = n.c[want];
            } else n = n.c[bit];
        }
        return ans;
    }
}
```

O(B) per insert/query where B=32 for int. Signed integer ordering and unsigned XOR interpretation should be clarified; for non-negative values, the usual pattern is straightforward.

Trie interview problems

• Implement Trie: insert/search/startsWith.

• Word Search II: board DFS + Trie pruning.

• Replace words using shortest root prefix.

• Maximum XOR of two numbers / subarray XOR.

• Count distinct substrings (often suffix structures or suffix automaton are more suitable).

• Autocomplete top-k suggestions; discuss caching and ranking.

4. Segment Tree: fundamentals

A Segment Tree stores information about intervals. Each node represents a segment [l,r], and combines its children's summaries. It is useful when array values change and range queries must remain fast.

Example: range sum

```
Array: [2, 1, 5, 3]
Root covers [0,3], sum=11
             [0,3]=11
             /      \
         [0,1]=3    [2,3]=8
          /  \       /  \
         2    1     5    3
```

Build O(n), point update O(log n), range sum O(log n), memory O(4n) in a recursive array implementation. Segment tree works for any associative merge operation; sum, min, max, gcd and many custom summaries are common.

Recursive Java sum segment tree

```
class SegTree {
    int n;
    long[] t;

    SegTree(int[] a) {
        n = a.length;
        t = new long[4 * Math.max(1, n)];
        if (n > 0) build(1, 0, n - 1, a);
    }

    void build(int v, int l, int r, int[] a) {
        if (l == r) { t[v] = a[l]; return; }
        int m = (l + r) >>> 1;
        build(v*2, l, m, a);
        build(v*2+1, m+1, r, a);
        t[v] = t[v*2] + t[v*2+1];
    }

    void set(int idx, long value) {
        if (n > 0) update(1, 0, n-1, idx, value);
    }

    void update(int v, int l, int r, int idx, long value) {
        if (l == r) { t[v] = value; return; }
        int m = (l+r) >>> 1;
        if (idx <= m) update(v*2, l, m, idx, value);
        else update(v*2+1, m+1, r, idx, value);
        t[v] = t[v*2] + t[v*2+1];
    }

    long query(int ql, int qr) {
        if (n == 0 || ql > qr) return 0;
        return query(1, 0, n-1, ql, qr);
    }

    long query(int v, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0; // no overlap
        if (ql <= l && r <= qr) return t[v]; // full cover
        int m = (l+r) >>> 1;
        return query(v*2,l,m,ql,qr)
             + query(v*2+1,m+1,r,ql,qr);
    }
}
```

Three query cases: no overlap → identity (0 for sum); full cover → stored summary; partial overlap → query children and merge. For minimum use +infinity as identity; for maximum use -infinity.

5. Segment Tree: advanced merges and variants

Generic monoid view

A segment tree needs an associative operation ⊕ and identity e. Associativity means (a⊕b)⊕c = a⊕(b⊕c). Sum, min, max, gcd, matrix multiplication and concatenation summaries can be used. If order matters, preserve left-to-right merge order.

Iterative segment tree

```
class IterSegSum {
    int n;
    long[] t;
    IterSegSum(long[] a) {
        n = a.length;
        t = new long[2*n];
        for (int i=0;i<n;i++) t[n+i]=a[i];
        for (int i=n-1;i>0;i--) t[i]=t[i<<1]+t[i<<1|1];
    }
    void set(int p,long value) {
        for (t[p+=n]=value; p>1; p>>=1)
            t[p>>1]=t[p]+t[p^1];
    }
    long sum(int l,int r) { // inclusive
        long ans=0;
        for (l+=n,r+=n+1;l<r;l>>=1,r>>=1) {
            if ((l&1)==1) ans+=t[l++];
            if ((r&1)==1) ans+=t[--r];
        }
        return ans;
    }
}
```

This compact iterative version is convenient for point updates and range queries. Empty arrays need care; for non-power-of-two n the iterative layout still works for this sum pattern.

More segment tree applications

• Maximum subarray sum: store total sum, best prefix, best suffix, best subarray; merge summaries carefully.

• Count zeros / ones, find kth zero: store counts and descend by left-child count.

• First element ≥ x: store maximum per segment and prune segments whose max < x.

• Coordinate compression: map large sparse values to sorted ranks before building.

• 2D segment tree: range queries in two dimensions, but memory and implementation complexity are high.

• Persistent segment tree: path-copy nodes to retain historical versions; useful for kth number in a subarray.

Common pitfalls

• Mixing inclusive and half-open intervals.

• Incorrect no-overlap identity (0 is not identity for minimum).

• Overflow: use long for sums if constraints can exceed int.

• Incorrect merge order for non-commutative summaries.

• Allocating 4*n without considering n=0 or integer overflow in capacity.

6. Lazy Propagation

If every update modifies a whole range, updating each leaf is too slow. Lazy propagation stores a pending operation on a segment node and delays pushing it to children until necessary.

Range add + range sum

Each node stores segment sum. `lazy[v]` means each element in this segment still needs +lazy[v]. If segment length is len, applying +d changes sum by d×len.

```
apply(v, len, d):
    tree[v] += d * len
    lazy[v] += d

push(v, leftLen, rightLen):
    if lazy[v] != 0:
        apply(leftChild, leftLen, lazy[v])
        apply(rightChild, rightLen, lazy[v])
        lazy[v] = 0
```

Java implementation: range add, range sum

```
class LazySegTree {
    int n;
    long[] sum, lazy;

    LazySegTree(int[] a) {
        n = a.length;
        sum = new long[4*Math.max(1,n)];
        lazy = new long[4*Math.max(1,n)];
        if (n>0) build(1,0,n-1,a);
    }
    void build(int v,int l,int r,int[] a) {
        if(l==r){sum[v]=a[l];return;}
        int m=(l+r)>>>1;
        build(v*2,l,m,a); build(v*2+1,m+1,r,a);
        sum[v]=sum[v*2]+sum[v*2+1];
    }
    void apply(int v,int len,long d) {
        sum[v]+=d*len; lazy[v]+=d;
    }
    void push(int v,int l,int r) {
        if(l==r || lazy[v]==0) return;
        int m=(l+r)>>>1;
        apply(v*2,m-l+1,lazy[v]);
        apply(v*2+1,r-m,lazy[v]);
        lazy[v]=0;
    }
    void add(int ql,int qr,long d) {
        if(n>0 && ql<=qr) add(1,0,n-1,ql,qr,d);
    }
    void add(int v,int l,int r,int ql,int qr,long d) {
        if(qr<l || r<ql) return;
        if(ql<=l && r<=qr){apply(v,r-l+1,d);return;}
        push(v,l,r);
        int m=(l+r)>>>1;
        add(v*2,l,m,ql,qr,d);
        add(v*2+1,m+1,r,ql,qr,d);
        sum[v]=sum[v*2]+sum[v*2+1];
    }
    long query(int ql,int qr) {
        if(n==0 || ql>qr) return 0;
        return query(1,0,n-1,ql,qr);
    }
    long query(int v,int l,int r,int ql,int qr) {
        if(qr<l || r<ql) return 0;
        if(ql<=l && r<=qr) return sum[v];
        push(v,l,r);
        int m=(l+r)>>>1;
        return query(v*2,l,m,ql,qr)
             + query(v*2+1,m+1,r,ql,qr);
    }
}
```

Range add and range sum are O(log n) each. Build O(n), memory O(n). The `push` operation preserves the parent summary by transferring pending work to children.

7. Lazy Propagation: range assignment and operation composition

Range set is more subtle than range add. A pending set overrides older pending adds/sets. Model lazy tags as functions and compose them in the correct order.

Range set + range sum

For a segment of length len, setting every value to x makes sum=x×len. Keep `hasSet[v]` separate from `setVal[v]` because setting to zero is valid. If later adding d, update setVal += d when a set tag exists; otherwise accumulate add tag.

```
applySet(v, len, x):
    sum[v] = x * len
    hasSet[v] = true
    setVal[v] = x
    add[v] = 0

applyAdd(v, len, d):
    sum[v] += d * len
    if hasSet[v]:
        setVal[v] += d
    else:
        add[v] += d

push(v):
    if hasSet[v]:
        applySet(children, child lengths, setVal[v])
        hasSet[v] = false
    if add[v] != 0:
        applyAdd(children, child lengths, add[v])
        add[v] = 0
```

Order matters: set then add is equivalent to a new set value; add then set is overwritten by the set. Always define tag composition before coding.

Other lazy operations

• Range multiply + sum: sum scales by multiplier; pending tags compose multiplicatively (with care for modulus/overflow).

• Range add + range minimum: node minimum increases by d; no segment-length multiplier is needed.

• Range assign + minimum: node minimum becomes assigned value.

• Affine updates x ■ a·x+b: compose tags as functions; useful for advanced problems.

When lazy propagation is not appropriate

If updates are irregular or the summary cannot be updated from a segment-level tag, a standard lazy segment tree may not work. Sometimes sqrt decomposition, an implicit treap, or a different invariant is better.

8. Fenwick Tree / Binary Indexed Tree

Fenwick Tree maintains prefix aggregates using a compact array. For sum, point add and prefix sum are O(log n), memory O(n). It is usually simpler and smaller than a segment tree when operations fit its model.

The lowbit trick

For positive integer i, `i & -i` extracts the least significant set bit. Fenwick node i covers a block of length lowbit(i).

```
i = 12 = 1100b
-i        two's complement
i & -i = 4 = 0100b
```

Java implementation (1-indexed)

```
class Fenwick {
    int n;
    long[] bit;
    Fenwick(int n) { this.n=n; bit=new long[n+1]; }

    void add(int index, long delta) {
        for (int i=index; i<=n; i+=i&-i)
            bit[i]+=delta;
    }

    long prefixSum(int index) {
        long ans=0;
        for (int i=index; i>0; i-=i&-i)
            ans+=bit[i];
        return ans;
    }

    long rangeSum(int left,int right) {
        return prefixSum(right)-prefixSum(left-1);
    }
}
```

Public indices are 1..n. `add(0, ...)` would loop forever because lowbit(0)=0; validate indices. For a 0-indexed input array, call add(i+1, delta).

Build in O(n)

```
for (int i=1; i<=n; i++) {
    bit[i] += a[i-1];
    int parent = i + (i & -i);
    if (parent <= n) bit[parent] += bit[i];
}
```

This linear build propagates each partial block to its Fenwick parent. It assumes the input values are already available.

9. Fenwick Tree: advanced patterns

Point assignment

Fenwick naturally supports adding a delta, not setting a value. Keep the original array: delta = newValue - oldValue, then add(index, delta).

Range add + point query

Use a difference array: adding x to [l,r] means add(l,+x), add(r+1,-x). The value at position p is prefixSum(p).

Range add + range sum

Use two Fenwick trees. If D is the difference array, prefix sum of original values through x is x·sum(D[1..x]) - sum((i-1)D[i], i=1..x). Maintain BIT1 for D and BIT2 for (i-1)D.

```
Range add [l,r] by v:
  B1.add(l, v)
  B1.add(r+1, -v)
  B2.add(l, v*(l-1))
  B2.add(r+1, -v*r)

prefixSum(x) = x * B1.sum(x) - B2.sum(x)
rangeSum(l,r) = prefixSum(r) - prefixSum(l-1)
```

Find smallest index with prefix sum ≥ target

If all frequencies are non-negative, binary lifting on Fenwick can find the first index whose prefix sum reaches target in O(log n). This is useful for order statistics over frequencies.

2D Fenwick

A 2D BIT supports point add and rectangle sum in O(log n · log m), using nested lowbit loops. Memory O(nm); coordinate compression is often needed for sparse large coordinates.

Fenwick vs Segment Tree

Feature	Fenwick	Segment Tree
Prefix sum / point add	Excellent, compact	Works, more general
Range sum / point update	Excellent	Excellent
Range min/max	Not naturally general	Natural
Custom associative merge	Limited	Very flexible
Range update	Difference/two-BIT patterns	Lazy propagation
Memory	O(n)	Usually O(4n)

10. Sparse Table

Sparse Table is for static arrays: build once, answer range queries quickly, but no efficient point updates. For idempotent operations (min, max, gcd), O(1) query is possible using overlapping power-of-two intervals.

Build

st[k][i] stores aggregate over interval [i, i+2^k-1]. `st[k][i] = op(st[k-1][i], st[k-1][i+2^(k-1)])`.

```
int K = 1;
while ((1 << K) <= n) K++;
int[][] st = new int[K][n];

for (int i=0;i<n;i++) st[0][i]=a[i];

for (int k=1;k<K;k++) {
    int len=1<<k, half=len>>1;
    for (int i=0;i+len<=n;i++)
        st[k][i]=Math.min(st[k-1][i],
                          st[k-1][i+half]);
}
```

O(1) range minimum query

```
int[] log = new int[n+1];
for (int i=2;i<=n;i++) log[i]=log[i/2]+1;

int queryMin(int l,int r) {
    int k=log[r-l+1];
    return Math.min(st[k][l],
                    st[k][r-(1<<k)+1]);
}
```

Two blocks overlap, but min is idempotent: min(x,x)=x, so overlap does not change the result. Same for max and gcd.

Why not for sum?

Sum is not idempotent. Overlapping blocks would count the overlap twice. For sum, use disjoint sparse table or another structure; ordinary sparse table gives O(log n) query by combining disjoint blocks.

Complexity

Operation	Complexity
Build	O(n log n)
Idempotent query	O(1)
Memory	O(n log n)
Update	Not efficient; rebuild may be needed

11. Disjoint Set Union (DSU / Union-Find)

DSU maintains a partition of elements into disjoint components. It supports finding a representative and merging two components. With path compression + union by size/rank, operations are nearly constant amortized: O(α(n)).

Core operations

```
class DSU {
    int[] parent, size;
    int components;

    DSU(int n) {
        parent=new int[n];
        size=new int[n];
        components=n;
        for(int i=0;i<n;i++){parent[i]=i;size[i]=1;}
    }

    int find(int x) {
        if(parent[x]!=x) parent[x]=find(parent[x]);
        return parent[x];
    }

    boolean union(int a,int b) {
        int ra=find(a), rb=find(b);
        if(ra==rb) return false;
        if(size[ra]<size[rb]) {
            int t=ra;ra=rb;rb=t;
        }
        parent[rb]=ra;
        size[ra]+=size[rb];
        components--;
        return true;
    }

    boolean connected(int a,int b) {
        return find(a)==find(b);
    }
}
```

Path compression flattens paths during find. Union by size attaches smaller component under larger. α(n), inverse Ackermann, grows so slowly it is ≤ a tiny constant for practical input sizes.

Applications

• Connected components in undirected graphs.

• Kruskal minimum spanning tree.

• Cycle detection in undirected graphs.

• Number of provinces / network connectivity.

• Offline connectivity queries after sorting edges by weight.

• Grid island merging; map each cell to an integer id.

12. DSU: advanced variants

DSU with parity / bipartite constraints

Store parity[x] = XOR parity from x to parent. During find, compress while updating parity. When adding an edge constraint, compare root parity to detect contradictions. This supports online bipartiteness checks in specialized formulations.

Rollback DSU

For offline dynamic connectivity, use DSU without path compression (union by size only) so every union changes O(1) parent/size fields. Push changes to a stack and restore to a saved stack size when rolling back. Combine with a segment tree over time intervals.

Potential DSU

Store a relative difference/potential from node to parent. Useful for constraints like x–y=d or coordinate differences. Derive how potentials change when roots are attached; sign conventions matter.

DSU limitations

• Classic DSU supports merges, not arbitrary component splits.

• Cannot answer arbitrary path queries inside a component.

• Path compression is incompatible with simple rollback because it changes many pointers.

• Use BFS/DFS for one-off component enumeration; DSU is ideal for repeated merges/connectivity.

13. Ordered Set and balanced BST concepts

An ordered set stores unique keys in sorted order and supports predecessor, successor, rank and range iteration. Java TreeSet/TreeMap use a Red-Black Tree internally, giving O(log n) search/insert/delete.

```
TreeSet<Integer> set = new TreeSet<>();
set.add(10); set.add(4); set.add(7);
int first = set.first();          // 4
Integer ceil = set.ceiling(6);    // 7
Integer floor = set.floor(6);     // 4
Integer higher = set.higher(7);   // 10
Integer lower = set.lower(7);     // 4
set.remove(7);
```

Java TreeSet does not expose kth element/rank in O(log n). For order statistics, use an augmented balanced tree, Treap, or coordinate compression + Fenwick.

AVL Tree

AVL maintains balance factor height(left)–height(right) in {-1,0,+1}. After insertion/deletion, rebalance using rotations.

Case	Shape	Fix
LL	Left child is left-heavy	Right rotation
RR	Right child is right-heavy	Left rotation
LR	Left child right-heavy	Left rotate child, then right rotate root
RL	Right child left-heavy	Right rotate child, then left rotate root

Red-Black Tree

Red-Black trees maintain color properties: root black, red nodes have black children, every root-to-null path has equal black height, and no two red nodes are adjacent. Rotations and recoloring maintain logarithmic height.

Treap

Treap combines BST order by key and heap order by random priority. Expected height is O(log n). Split and merge are elegant and support implicit sequence treaps.

Order-statistic tree

Augment each node with subtree size. This supports kth smallest and rank(key) in O(log n) in a balanced tree. Every rotation/update must recompute metadata.

14. Monotonic Stack

A monotonic stack maintains elements in increasing or decreasing order. Each element is pushed once and popped once, so many “next greater/smaller” problems are O(n), not O(n^2).

Next greater element to right

```
int[] nextGreater(int[] a) {
    int n=a.length;
    int[] ans=new int[n];
    Arrays.fill(ans,-1);
    Deque<Integer> st=new ArrayDeque<>(); // indices

    for(int i=0;i<n;i++) {
        while(!st.isEmpty() && a[st.peek()]<a[i]) {
            ans[st.pop()]=a[i];
        }
        st.push(i);
    }
    return ans;
}
```

The stack holds indices whose next greater value has not been found. When current value is larger, it resolves all smaller values at the top.

Next smaller / previous greater

Change the comparison direction and scan direction. Decide strict vs non-strict comparisons carefully when duplicates exist.

Largest rectangle in histogram

```
long largestRectangle(int[] h) {
    Deque<Integer> st=new ArrayDeque<>();
    long best=0;
    for(int i=0;i<=h.length;i++) {
        int cur=(i==h.length)?0:h[i];
        while(!st.isEmpty() && h[st.peek()]>=cur) {
            int height=h[st.pop()];
            int left=st.isEmpty()?0:st.peek()+1;
            int width=i-left;
            best=Math.max(best,(long)height*width);
        }
        st.push(i);
    }
    return best;
}
```

A sentinel height 0 flushes remaining bars. For input heights that may include 0, this remains valid. Use long for area.

15. Monotonic Queue / Deque

A monotonic deque maintains candidate indices for a sliding-window minimum or maximum. Expired indices are removed from the front; dominated values are removed from the back.

Sliding window maximum

```
int[] maxSlidingWindow(int[] a,int k) {
    if(k<=0 || k>a.length) return new int[0];
    int[] ans=new int[a.length-k+1];
    Deque<Integer> dq=new ArrayDeque<>();

    for(int i=0;i<a.length;i++) {
        while(!dq.isEmpty() && dq.peekFirst()<=i-k)
            dq.pollFirst();

        while(!dq.isEmpty() && a[dq.peekLast()]<=a[i])
            dq.pollLast();

        dq.offerLast(i);
        if(i>=k-1) ans[i-k+1]=a[dq.peekFirst()];
    }
    return ans;
}
```

Each index enters and leaves at most once: O(n) time, O(k) space. For minimum, reverse the comparison.

Where monotonic queues appear

• Sliding window min/max.

• Shortest subarray with sum at least K (often monotonic prefix-sum deque).

• DP optimization with a moving range of candidates.

• Stock span, daily temperatures, subarray contribution counting.

16. Suffix Array

A suffix array stores the starting indices of all suffixes of a string in lexicographic order. It supports substring search, longest repeated substring and LCP-based queries.

For string `banana`, suffixes in sorted order begin at indices [5,3,1,0,4,2] (depending on sentinel convention).

Prefix-doubling idea

• Initially sort suffixes by first character.

• At step k, sort suffixes by pair (rank[i], rank[i+2^k]).

• Assign new equivalence classes; double k until all ranks are unique.

• Use counting/radix sorting for O(n log n) construction; simple comparison sorting often costs O(n log² n).

LCP array and Kasai

LCP[i] is the common-prefix length of suffixArray[i] and suffixArray[i–1]. Kasai computes all LCP values in O(n) after suffix array and inverse ranks are known.

Java prefix-doubling reference implementation

```
int[] suffixArray(String s) {
    int n=s.length();
    Integer[] sa=new Integer[n];
    int[] rank=new int[n], next=new int[n];
    for(int i=0;i<n;i++){sa[i]=i;rank[i]=s.charAt(i);}

    for(int len=1;len<n;len*=2) {
        final int L=len;
        Arrays.sort(sa,(a,b)->{
            if(rank[a]!=rank[b]) return Integer.compare(rank[a],rank[b]);
            int ra=a+L<n?rank[a+L]:-1;
            int rb=b+L<n?rank[b+L]:-1;
            return Integer.compare(ra,rb);
        });

        next[sa[0]]=0;
        for(int i=1;i<n;i++) {
            int a=sa[i-1], b=sa[i];
            boolean diff=rank[a]!=rank[b]
                || (a+L<n?rank[a+L]:-1)!=(b+L<n?rank[b+L]:-1);
            next[b]=next[a]+(diff?1:0);
        }
        int[] tmp=rank; rank=next; next=tmp;
        if(rank[sa[n-1]]==n-1) break;
        if(len>n/2) break; // avoid int overflow in doubling
    }
    int[] out=new int[n];
    for(int i=0;i<n;i++)out[i]=sa[i];
    return out;
}
```

This educational version uses object sorting and is O(n log² n) due to comparison sort each round. High-performance CP implementations use counting/radix sorting to achieve O(n log n).

17. LCP with Kasai and suffix applications

```
int[] kasai(String s,int[] sa) {
    int n=s.length();
    int[] rank=new int[n], lcp=new int[n];
    for(int i=0;i<n;i++)rank[sa[i]]=i;
    int k=0;
    for(int i=0;i<n;i++) {
        int r=rank[i];
        if(r==0) continue;
        int j=sa[r-1];
        while(i+k<n && j+k<n && s.charAt(i+k)==s.charAt(j+k)) k++;
        lcp[r]=k;
        if(k>0) k--;
    }
    return lcp;
}
```

Applications

• Longest repeated substring = maximum LCP.

• Number of distinct substrings = n(n+1)/2 – sum(LCP), for a string of length n.

• Substring search: binary search pattern among suffixes, then compare characters.

• Longest common substring of two strings: concatenate with unique separator and inspect adjacent suffixes from different sources.

• Lexicographic substring ranking and text indexing.

Suffix Array vs Trie vs Suffix Automaton

Structure	Best at	Trade-off
Trie	Dictionary/prefix queries	Memory can be high
Suffix Array	Static text suffix ordering	Build/index complexity
Suffix Automaton	Distinct substrings, end-position equivalence	Subtle state/link invariants
Rolling Hash	Fast probabilistic substring equality	Collision risk; verify if exactness required

18. LRU Cache

Least Recently Used evicts the key that has not been accessed for the longest time. Required O(1) get and put are achieved by HashMap + doubly linked list.

Design

• HashMap maps key → list node.

• List head is most recently used; tail is least recently used.

• get: lookup, move node to front.

• put existing key: update value, move to front.

• put new key: insert front; if over capacity, remove tail and map entry.

```
class LRUCache {
    class Node {
        int key,value;
        Node prev,next;
        Node(int k,int v){key=k;value=v;}
    }
    int cap;
    Map<Integer,Node> map=new HashMap<>();
    Node head=new Node(0,0), tail=new Node(0,0);

    LRUCache(int capacity) {
        cap=capacity; head.next=tail; tail.prev=head;
    }
    void remove(Node n) {
        n.prev.next=n.next; n.next.prev=n.prev;
    }
    void addFront(Node n) {
        n.next=head.next; n.prev=head;
        head.next.prev=n; head.next=n;
    }
    void touch(Node n) { remove(n); addFront(n); }

    int get(int key) {
        Node n=map.get(key);
        if(n==null)return -1;
        touch(n); return n.value;
    }
    void put(int key,int value) {
        if(cap<=0)return;
        Node n=map.get(key);
        if(n!=null){n.value=value;touch(n);return;}
        n=new Node(key,value);map.put(key,n);addFront(n);
        if(map.size()>cap) {
            Node old=tail.prev;remove(old);map.remove(old.key);
        }
    }
}
```

Expected O(1) get/put. For production use, handle concurrency, generic types, null values, capacity semantics and memory limits explicitly.

19. LFU Cache

Least Frequently Used evicts the key with the smallest access frequency. When frequencies tie, evict the least recently used among those keys.

O(1) design idea

• key → node map.

• frequency → doubly linked list of nodes, each list ordered by recency.

• Each node stores key, value, frequency.

• Track minFrequency.

• get/put existing: remove from old frequency list, increment frequency, add to new list front.

• Eviction: remove tail of minFrequency list.

LFU is more complex than LRU because every successful access changes frequency. New keys typically start at frequency 1 and reset minFrequency to 1.

Implementation outline

```
class LFUCache {
    class Node {
        int key,value,freq=1;
        Node prev,next;
        Node(int k,int v){key=k;value=v;}
    }
    class DList {
        Node head=new Node(0,0), tail=new Node(0,0);
        int size=0;
        DList(){head.next=tail;tail.prev=head;}
        void addFront(Node x){
            x.next=head.next;x.prev=head;
            head.next.prev=x;head.next=x;size++;
        }
        void remove(Node x){
            x.prev.next=x.next;x.next.prev=x.prev;size--;
        }
        Node removeLast(){
            if(size==0)return null;
            Node x=tail.prev;remove(x);return x;
        }
    }
    int cap,minFreq=0;
    Map<Integer,Node> nodes=new HashMap<>();
    Map<Integer,DList> lists=new HashMap<>();
    LFUCache(int c){cap=c;}

    void touch(Node x){
        int f=x.freq;
        DList old=lists.get(f);old.remove(x);
        if(f==minFreq && old.size==0)minFreq++;
        x.freq++;
        lists.computeIfAbsent(x.freq,z->new DList()).addFront(x);
    }
    int get(int key){
        Node x=nodes.get(key);if(x==null)return -1;
        touch(x);return x.value;
    }
    void put(int key,int value){
        if(cap==0)return;
        if(nodes.containsKey(key)){
            Node x=nodes.get(key);x.value=value;touch(x);return;
        }
        if(nodes.size()==cap){
            Node ev=lists.get(minFreq).removeLast();
            nodes.remove(ev.key);
        }
        Node x=new Node(key,value);nodes.put(key,x);
        lists.computeIfAbsent(1,z->new DList()).addFront(x);
        minFreq=1;
    }
}
```

Expected O(1) get/put under hash map assumptions. This is a standard LFU+LRU tie-break design. Production code should add tests for capacity 0, repeated accesses, update-existing-key and frequency-list empty transitions.

20. Additional important structure: Heap / Priority Queue

A binary heap is a complete binary tree stored in an array. In a min-heap, parent ≤ children. Java PriorityQueue is a min-heap by default.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(9); pq.offer(2); pq.offer(6);
int x = pq.poll(); // 2

PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>(Comparator.reverseOrder());
```

Operation	Binary heap
Peek root	O(1)
Insert	O(log n)
Remove root	O(log n)
Build heap	O(n)
Search arbitrary value	O(n)

Interview patterns

• Top K elements, kth largest/smallest.

• Merge K sorted lists/arrays.

• Median from data stream: max-heap lower half + min-heap upper half.

• Dijkstra and Prim (priority queue).

• Task scheduling and interval selection.

21. Sqrt Decomposition

Split an array into blocks of size approximately √n. Store a summary per block. Query scans partial boundary blocks and uses summaries for full blocks.

For range sum with point updates: update one element O(1) plus block sum update; query O(√n). Range add can use block lazy values. It is simpler than a segment tree and useful when constraints or operation mix favor it.

```
int blockSize=(int)Math.sqrt(n)+1;
long[] blockSum=new long[(n+blockSize-1)/blockSize];

long query(int l,int r) {
    long ans=0;
    while(l<=r && l%blockSize!=0) ans+=a[l++];
    while(l+blockSize-1<=r) {
        ans+=blockSum[l/blockSize]; l+=blockSize;
    }
    while(l<=r) ans+=a[l++];
    return ans;
}
```

Mo's algorithm

Offline range queries: sort queries by left block and right endpoint in alternating order. Maintain one current interval and add/remove elements as endpoints move. Often O((n+q)√n) movement for frequency-based answers, depending on add/remove cost.

22. Persistent Segment Tree

A persistent structure keeps old versions after updates. A persistent segment tree path-copies only nodes along the updated root-to-leaf path; untouched subtrees are shared.

• Build initial root version.

• On update at position p, create new nodes only on the affected path.

• Keep old root references for previous versions.

• Range queries can target any saved version.

• Memory per point update O(log n), not O(n), with path copying.

Applications: kth smallest in subarray using prefix versions; historical queries; versioned arrays. The implementation requires careful node allocation and immutable references.

Persistent kth in a subarray

Build a prefix version root[i] containing frequencies of values in first i elements (after coordinate compression). For query [l,r], subtract counts in root[l–1] from root[r] and descend by left-subtree count difference to find kth smallest.

23. Wavelet Tree / Wavelet Matrix

A Wavelet Tree recursively partitions values by bits or value ranges, storing prefix counts that tell how many elements go to the left partition. It supports kth smallest, count $\leq x$, and range frequency queries in $O(\log \sigma)$ for alphabet/value domain σ.

It is powerful for static range order statistics and can be more memory efficient than persistent segment trees in some settings. It is less common in general product interviews but useful in advanced CP.

When to choose

• Static array + many kth-in-range queries: wavelet tree or persistent segment tree.

• Dynamic point updates + range kth: more complex dynamic structures; ordinary segment tree of multisets or balanced trees may be used.

• Small value domain: coordinate compression + persistent frequency segment tree.

24. Tree flattening: Euler Tour, HLD and LCA

Euler Tour / DFS order

A DFS entry time tin[u] and exit time tout[u] make each subtree a contiguous interval [tin[u], tout[u]) in preorder. Then subtree sum/update can be handled by Fenwick or segment tree over the flattened array.

```
int timer=0;
void dfs(int u,int parent) {
    tin[u]=timer;
    flat[timer++]=value[u];
    for(int v: graph[u]) if(v!=parent) dfs(v,u);
    tout[u]=timer;
}
// subtree u corresponds to flat[tin[u] .. tout[u)-1]
```

Heavy-Light Decomposition (HLD)

HLD decomposes a tree path into O(log n) heavy-chain intervals. Flatten each chain into an array and use a segment tree for path queries/updates. Typical path query O(log² n) with a segment tree, depending on operation and implementation.

LCA

For static trees, binary lifting preprocesses 2^k-th ancestors in O(n log n) and answers LCA in O(log n). Euler Tour + RMQ can answer O(1) after preprocessing. These are tree-query structures, distinct from BST LCA.

25. Bitsets, Bloom Filters and Hashing variants

Bitset

A bitset stores boolean flags compactly and supports fast bitwise operations. Java `BitSet` supports set, clear, get, and, or, xor. Useful for dense subsets, sieve, reachability and DP.

Bloom Filter

A Bloom filter is a probabilistic membership structure. It can return false positives but not false negatives (assuming correct insert/query use). It stores bit positions from multiple hashes; no deletion in a standard Bloom filter.

HashMap internals

Hash tables provide expected O(1) lookup/insert/delete, but collisions, resizing, hash quality and adversarial keys matter. Java HashMap uses buckets and may treeify collision-heavy buckets under implementation-specific conditions.

When not to use hashing

• Need sorted order or predecessor/successor: ordered tree.

• Need prefix matching: Trie.

• Need deterministic worst-case bound: balanced tree or sorted array, depending on updates.

• Need range aggregates: Fenwick/segment tree/sparse table.

26. Advanced pattern selection guide

Problem statement clue	Think of
Words share prefixes; dictionary	Trie
Max XOR / bitwise prefix	Binary Trie
Dynamic range sum/min/max	Segment Tree
Range update and range query	Lazy Segment Tree
Point add and prefix sum	Fenwick Tree
Static range min/gcd	Sparse Table
Merge components / cycle detection	DSU
Sorted dynamic keys / rank	Balanced BST / Treap
Next greater/smaller	Monotonic Stack
Window min/max	Monotonic Deque
Static text suffixes / repeated substrings	Suffix Array + LCP
Evict least recently used	HashMap + Doubly Linked List
Evict least frequently used	HashMaps + frequency lists
Top K / repeatedly extract min	Heap
Range query with simpler block structure	Sqrt Decomposition
Historical array versions	Persistent Segment Tree
Kth in subarray	Persistent Segment Tree / Wavelet Tree
Subtree queries	Euler Tour + BIT/Segment Tree
Tree path queries	HLD + Segment Tree

Decision questions to ask yourself

• Are updates online or can queries be sorted offline?

• Is the array static or dynamic?

• Is the operation invertible? (Fenwick range sum uses prefix subtraction.)

• Is the operation idempotent? (Sparse Table O(1) overlap needs this.)

• Can a segment summary be merged associatively?

• Can a lazy update be applied to a summary without visiting leaves?

• Do I need sorted order, rank, predecessor or successor?

• Do I need exact answers or are probabilistic structures acceptable?

27. Complexity master sheet

Structure	Build	Query / operation	Update
Trie	O(total chars)	O(L) search/prefix	O(L) insert/delete
Segment Tree	O(n)	O(log n) range	O(log n) point
Lazy Segment Tree	O(n)	O(log n) range	O(log n) range
Fenwick	O(n) build	O(log n) prefix/range	O(log n) point add
Sparse Table	O(n log n)	O(1) idempotent	Not efficient
DSU	O(n) init	O(α(n)) find	O(α(n)) union amortized
Balanced BST	O(n) from sorted	O(log n)	O(log n)
Monotonic stack	O(n) scan	—	O(1) amortized push/pop
Monotonic deque	O(n) scan	O(1) front	O(1) amortized
Suffix Array	O(n log n) typical	LCP/search dependent	Static
LRU/LFU	—	Expected O(1)	Expected O(1)
Heap	O(n) heapify	O(1) peek	O(log n) insert/pop
Sqrt decomposition	O(n)	O(√n) typical	O(1) point / block
Persistent SegTree	O(n) initial	O(log n)	O(log n) new version
Wavelet Tree	O(n log σ) typical	O(log σ)	Static variants

These are typical bounds under standard assumptions. Actual complexity depends on the operation, merge type, balancing guarantee, alphabet/value domain and implementation.

28. High-level interview and competitive programming problem bank

Level	Problem	Core technique
Foundation	Implement Trie	Prefix tree
Foundation	Range Sum Query Mutable	Fenwick / Segment Tree
Foundation	Range Minimum Query static	Sparse Table
Foundation	Number of Islands II	DSU
Foundation	Next Greater Element	Monotonic Stack
Foundation	Sliding Window Maximum	Monotonic Deque
Core	Range Add + Range Sum	Lazy Segment Tree / 2 BIT
Core	Count inversions	Fenwick / merge sort
Core	Kth smallest in subarray	Persistent tree / Wavelet
Core	Maximum XOR pair	Binary Trie
Core	LRU Cache	HashMap + DLL
Core	LFU Cache	Frequency lists
Core	Largest Rectangle Histogram	Monotonic Stack
Core	Longest Repeated Substring	Suffix Array + LCP
Core	Kruskal MST	DSU + sorting
Advanced	Offline dynamic connectivity	Rollback DSU + time segment tree
Advanced	Subtree sum updates	Euler Tour + Fenwick/SegTree
Advanced	Tree path maximum/sum	HLD + Segment Tree
Advanced	Persistent kth range query	Persistent Segment Tree
Advanced	Range affine updates	Lazy tag composition
Advanced	Online order statistics	Augmented balanced BST / Treap

29. Interview explanation templates

Explain Segment Tree

“I divide the array into intervals. Each node stores an associative summary of its interval. A range query visits only O(log n) boundary paths and fully covered nodes; a point update recomputes summaries on one root-to-leaf path.”

Explain Lazy Propagation

“I store pending updates at segment nodes. If a node is fully covered, I update its summary and record a lazy tag instead of descending. Before partially visiting children, I push the tag down. This makes range updates logarithmic.”

Explain Fenwick

“The lowbit of an index identifies the block represented by that BIT cell. Prefix sum walks downward by subtracting lowbit; point add walks upward by adding lowbit.”

Explain DSU

“Each component is represented by a root. Find returns the root; union attaches one root to another. Path compression and union by size make operations inverse-Ackermann amortized.”

Explain Monotonic Stack

“The stack keeps unresolved indices in monotonic value order. A new element pops every element it resolves. Each index is pushed and popped at most once, so total work is linear.”

Explain LRU

“The map gives O(1) node lookup; the doubly linked list gives O(1) movement and eviction. Head is most recent and tail is least recent.”

30. Mistakes that cost interview points

• Choosing a segment tree when Fenwick is enough—or choosing Fenwick when custom merge is needed.

• Using Sparse Table for updates without acknowledging rebuild cost.

• Applying overlapping sparse-table blocks to sum (not idempotent).

• Using 0-index Fenwick update with lowbit(0), causing an infinite loop.

• Not pushing lazy tags before partial descent.

• Composing lazy set/add tags in the wrong order.

• Using DSU for a problem that needs component splits or path queries.

• Using a monotonic stack but storing values when duplicate indices are required.

• Failing to remove expired indices from a sliding-window deque.

• Building suffix array with comparison sort and claiming $O(n \log n)$ instead of $O(n \log^2 n)$.

• LRU list pointer bugs: forgetting map removal on eviction.

• LFU: failing to update minFrequency after the last node leaves a frequency bucket.

• Integer overflow in range sums, histogram areas, XOR assumptions or pair sums.

• Ignoring empty inputs, k invalid, capacity zero, duplicate values and out-of-range indices.

Testing discipline

• Test n=0 and n=1 where allowed.

• Test all equal values, strictly increasing and decreasing inputs.

• Test negative values and large values.

• Test full-range, single-index and empty-range queries.

• Compare advanced structure answers against a brute-force implementation on random small inputs.

• Test repeated updates and overlapping lazy operations in different orders.

• Test duplicate keys and tie-breaking rules explicitly.

31. Suggested 8-week mastery plan

Week	Focus	Deliverable
1	Foundations, Trie, binary Trie	Implement dictionary + max XOR
2	Segment Tree basics and variants	Sum/min/max + kth zero
3	Lazy propagation	Range add/sum; range set/add composition
4	Fenwick and Sparse Table	BIT variants + static RMQ
5	DSU and rollback concept	Kruskal + offline connectivity
6	Balanced BST, Treap, monotonic structures	Order statistics + stack/deque patterns
7	Suffix Array, LCP, LRU/LFU	Implement string index and caches
8	Persistent tree, HLD, Mo's, mixed practice	Solve advanced timed sets

What to master first for interviews

Highest return: Trie, Segment Tree basics, Fenwick, DSU, monotonic stack/deque, heaps, LRU cache, balanced BST concepts. For competitive programming depth, add lazy propagation, sparse table, suffix array, rollback DSU, persistent segment tree, HLD and Mo's algorithm.

Final target

You should be able to identify the right structure from operation constraints, state its invariant, implement its core operations in Java, prove why it works, and explain time/space complexity. That is the real mastery—not memorizing a giant code dump.